

Complete Technical Interview Preparation Guide - Girish

This document is tailored specifically for your AI + Software Engineering interviews based on your resume, internship, backend systems knowledge, AI/ML experience, DevOps skills, and full-stack projects.

1. Strong Interview Introduction

Hi, I'm Girish, a Data Science undergraduate with strong interest in AI-driven software engineering, backend systems, and applied machine learning.

Recently, I worked as an AI/Software Development Intern at DrugParadigm, where I contributed to a reinforcement learning-based molecular generation system inspired by AstraZeneca's PROTAC Invent framework.

My work mainly focused on backend APIs, GPU-accelerated tool integrations, and improving reliability of computational workflows. I integrated open-source alternatives like ROSHAMBO and OpenMM into the reinforcement learning scoring pipeline and implemented asynchronous APIs with retry and fallback mechanisms for reliability.

Apart from that, I've built multiple full-stack and AI-powered projects including EmotiLearn for dyslexic children, FixMyHyd for civic issue reporting using AI workflows, and Nivasa, a full-stack booking platform.

I also have strong interest in scalable backend systems, DevOps, and system design concepts. I've worked with AWS, Docker, Kubernetes, Terraform, Redis, CI/CD pipelines, JWT authentication, rate limiting, caching, and asynchronous processing.

Alongside development, I actively practice DSA and have solved 350+ LeetCode problems. I enjoy building systems where AI, backend engineering, and real-world problem solving come together.

2. Internship Technical Questions & Answers

Q1. Explain the architecture of Protech Invent.

Answer:

The system was a reinforcement learning-based molecular generation framework. The user provides ligand SMILES strings, 3D structures, scoring constraints, and docking protein files. Internally, the framework uses a pretrained LSTM model trained on ChEMBL datasets.

During RL iterations, the agent model generates candidate molecules which are evaluated using the reinvent-scoring pipeline. Rewards are calculated using components such as shape similarity, ADMET properties, docking, conformer generation, and energy minimization. Based on rewards, the agent model updates iteratively.

Q2. What exactly was your contribution?

Answer:

I worked mainly on backend integrations and computational scoring workflows. I integrated ROSHAMBO for GPU-accelerated shape similarity scoring and OpenMM for energy minimization. I also developed asynchronous Flask APIs, implemented retry/fallback mechanisms, improved reliability, and benchmarked outputs against the original implementation.

Q3. Why async APIs?

Answer:

The molecular computations were computationally expensive and long-running. Async APIs improved concurrency, scalability, responsiveness, and reliability by preventing blocking operations and enabling retry handling.

Q4. What challenges did you face?

Answer:

Understanding the large research-oriented codebase and setting up ROSHAMBO with GPU/Linux compatibility were major challenges. I also had to manage CLI parameters, async execution issues, and fault-tolerant retry handling.

Q5. What did you learn?

Answer:

I learned reinforcement learning workflows, molecular modeling concepts, GPU tooling, async backend engineering, TensorBoard monitoring, Optuna hyperparameter tuning, Linux environments, and scalable backend API design.

3. Full Stack Interview Questions

Q1. SQL vs NoSQL?

Answer:

SQL databases are relational and best for structured data, transactions, joins, and consistency. NoSQL databases are flexible, schema-less, and better for scalability and high-throughput distributed systems.

Q2. When would you prefer MongoDB over MySQL?

Answer:

I would prefer MongoDB when data structure changes frequently, rapid development is required, or the application handles semi-structured/unstructured data.

Q3. What is indexing?

Answer:

Indexing improves query performance by reducing full table scans. It allows databases to locate rows efficiently.

Q4. Query optimization techniques?

Answer:

Using indexes, avoiding SELECT *, optimizing joins, pagination, caching frequent queries, denormalization when needed, and analyzing execution plans.

Q5. What is normalization?

Answer:

Normalization reduces redundancy and improves consistency by organizing data into multiple related tables.

Q6. Explain caching.

Answer:

Caching stores frequently accessed data temporarily to reduce database load and improve response times. Examples include Redis caching, CDN caching, browser caching, and API response caching.

Q7. Different types of caching?

Answer:

Browser cache, CDN cache, application cache, Redis/in-memory cache, database query cache, and reverse proxy cache.

Q8. Why Redis?

Answer:

Redis provides extremely fast in-memory access, making it useful for caching, session storage, rate limiting, Celery task queues, and real-time systems.

Q9. JWT vs Sessions?

Answer:

JWT is stateless and scalable for distributed systems, while sessions are stateful and stored on the server.

Q10. What is idempotency?

Answer:

An operation is idempotent if repeating it multiple times produces the same result.

Q11. What is rate limiting?

Answer:

Rate limiting restricts the number of requests from a client within a time window to prevent abuse and improve stability.

4. DevOps & Cloud Questions

Q1. What is Docker?

Answer:

Docker is a containerization platform that packages applications with dependencies to ensure consistent execution across environments.

Q2. Difference between Docker and Virtual Machines?

Answer:

Docker containers share the host OS kernel and are lightweight, whereas VMs run separate operating systems and consume more resources.

Q3. What is Kubernetes?

Answer:

Kubernetes is an orchestration platform used for deploying, scaling, and managing containerized applications.

Q4. What is Terraform?

Answer:

Terraform is Infrastructure as Code (IaC) tooling used to provision and manage cloud infrastructure declaratively.

Q5. Explain your AWS experience.

Answer:

I deployed applications using AWS services like EC2, S3, Lambda, and SNS. I also used Terraform for infrastructure automation.

Q6. What is CI/CD?

Answer:

CI/CD automates code integration, testing, and deployment to improve software delivery speed and reliability.

5. Python Questions

Q1. Difference between list and tuple?

Answer:

Lists are mutable while tuples are immutable.

Q2. What are decorators?

Answer:

Decorators allow modification of function behavior without changing the original function.

Q3. What is multithreading vs multiprocessing?

Answer:

Multithreading shares memory and is useful for I/O-bound tasks, while multiprocessing uses separate processes and is useful for CPU-bound tasks.

Q4. What is async programming?

Answer:

Async programming allows non-blocking execution of tasks, improving concurrency and scalability.

Q5. What are generators?

Answer:

Generators yield values lazily using the yield keyword and reduce memory usage.

6. SQL Coding Questions

Second Highest Salary:

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Nth Highest Salary:

```
SELECT DISTINCT salary
FROM employees e1
WHERE N-1 = (
SELECT COUNT(DISTINCT salary)
FROM employees e2
WHERE e2.salary > e1.salary
);
```

Find duplicate emails:

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Top 3 salaries department-wise:

```
SELECT *
FROM (
  SELECT department, salary,
  DENSE_RANK() OVER(PARTITION BY department ORDER BY salary DESC) as rnk
  FROM employees
) t
WHERE rnk <= 3;
```

7. System Design Concepts

Topics you must prepare deeply:

- API Gateway
- Load Balancing
- Redis Caching
- Circuit Breaker Pattern
- Retry Mechanisms
- Database Sharding
- CDN
- Message Queues
- WebSockets
- Rate Limiting
- Microservices
- Event-driven Architecture
- Idempotency
- Horizontal vs Vertical Scaling

Most important:

Always explain:

1. Problem
2. Bottleneck
3. Solution
4. Tradeoffs

8. DSA Questions Commonly Asked

Most common interview topics:

- Arrays
- Strings
- Sliding Window
- Two Pointers
- HashMaps
- Stack & Queue
- Binary Search
- Trees

- BFS/DFS
- Dynamic Programming

Important LeetCode Questions:

- Two Sum
- Longest Substring Without Repeating Characters
- Valid Parentheses
- Merge Intervals
- LRU Cache
- Binary Tree Level Order Traversal
- Number of Islands
- Kth Largest Element

9. Behavioral Questions

Q1. Tell me about a difficult challenge.

Answer:

One major challenge was understanding a large research-oriented AI codebase involving reinforcement learning, GPU workflows, and external computational tools. I approached it systematically by tracing workflows, using documentation, experimenting locally, and validating outputs incrementally.

Q2. Why should we hire you?

Answer:

I bring a combination of backend engineering, AI integration, full-stack development, and strong problem-solving skills. I enjoy understanding systems deeply and building scalable, reliable applications.

Q3. What are your strengths?

Answer:

Independent learning, understanding large systems, backend architecture, problem-solving, and persistence.

Q4. What is your weakness?

Answer:

Initially, I used to go too deep technically while explaining concepts. Over time, I learned to structure explanations based on audience and business context.

10. Interview Strategy

1. Speak slowly and clearly.
2. Start high-level, then go technical.
3. Never dump all details at once.
4. Explain architecture before implementation.
5. During coding:
 - Clarify problem
 - Discuss brute force
 - Optimize
 - Explain complexity
 - Dry run
6. Show ownership in projects.

7. Show learning mindset.

8. If you don't know something:

"I haven't implemented that directly yet, but based on my understanding..."